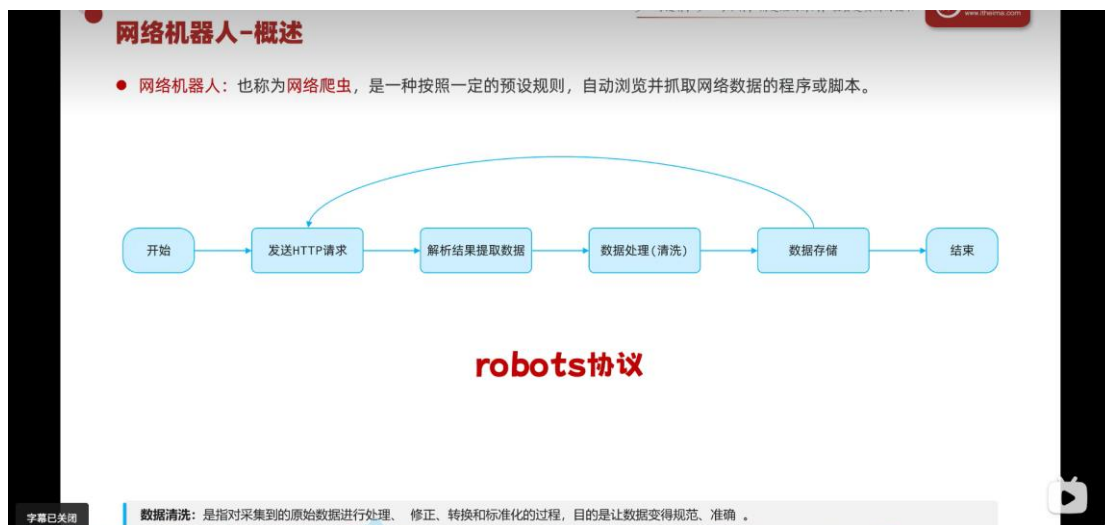


1.0 概述与 robots 协议



网络机器人-合规性(robots)

● robots协议也称为爬虫协议、爬虫规则，是指网站根目录下存放的一份文本文件robots.txt，用于告诉爬虫哪些页面可以抓取，哪些页面不能抓取。（君子协议）

```
User-agent: *
Disallow: /wp-admin/
Allow: /wp-admin/admin-ajax.php
Sitemap: https://www.tiobe.com/sitemap_index.xml
Crawl-delay: 5
```

通用规则(针对所有爬虫)

```
User-agent: Wandoujia Spider
Disallow: /
```

特定规则(针对豌豆荚爬虫)

```
User-agent: Mediapartners-Google
Disallow: /subject_search
Disallow: /amazon_search
Disallow: /search
Disallow: /group/search
```

特定规则(针对谷歌广告爬虫)

User-Agent: 用户代理，通过该请求头确认爬虫的类型

Disallow: 禁止访问的资源

Allow: 允许访问的资源

Sitemap: 网站地图，帮助爬虫更高效地获取网站内容

Crawl-delay: 爬取间隔时间，避免频繁访问造成网站压力过大

即网站关于爬虫的规定；

1.1 py 爬虫入门程序

步骤 获取TIOBE编程语言排行榜单

1. 查看TIOBE网站的robots.txt文件，明确资源获取的规则
2. 安装requests库，用于发送网络请求 (`pip install requests`)
3. 编写python代码，访问TIOBE网站，获取数据

requests

`https://www.tiobe.com/tiobe-index`

所涉及到的三步操作

```
import requests

#定义目标URL
target_url="https://www.tiobe.com/tiobe-index/"
#发送get请求,接收数据
response=requests.get(target_url)
#打印 数据
print(response.text)
```

打印出来整个html数据;

1.2 前端网页结构(为网页解析做准备)

前端网页结构

- 一个网页是由三个部分组成的，分别是：HTML、CSS、JS (JavaScript) 。
 - HTML：超文本标记语言，由一堆预设的标签(<h1>一级标题</h1>)构成。HTML负责网页的结构（页面元素和内容）
 - CSS：层叠样式表。CSS负责网页的表现（页面元素的外观、位置等样式，如颜色、大小等）
 - JS：全称为JavaScript，简称JS。负责网页的行为（交互效果）

The diagram illustrates the components of a web page. It shows a code editor with HTML, CSS, and JavaScript code. The HTML code defines the structure, including a title, navigation, and footer. The CSS code defines the styling, such as font size and color. The JavaScript code defines the behavior, such as changing the title color on mouseover. The diagram is divided into three sections: CSS control (样式), HTML control (结构), and JS control (行为).

CSS控制页面的样式

HTML控制页面的结构(内容)

JS控制页面的动作(行为)

所以接下来呢我们再来介绍一下这个HTML

前端网页结构-HTML

- HTML(HyperText Markup Language)：超文本标记语言。
 - 超文本：超越了文本的限制，比普通文本更强大。除了文字信息，还可以定义图片、音频、视频等内容。
 - 标记语言：由标签 "<标签名>" 构成的语言
 - HTML标签都是预定义好的。例如：使用<h1>展示标题，使用展示图片，使用<video>展示视频。
 - HTML代码直接在浏览器中运行，HTML标签由浏览器解析。

```
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>仙逆人物志 - 修真世界</title>
</head>
<body>
  <h1>Python</h1>
  <p>一门简介、快速、易用的编程语言。</p>
  <a href="https://www.itcast.cn">传智教育-黑马程序员</a>
</body>
</html>
```

开始标签 属性 结束标签



1.3 网页解析

网页解析

- 网页解析指的是从原始HTML文档中提取数据的过程，也是网络爬虫的关键步骤，从一堆标签文本中提取出需要的数据。

```
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>仙逆人物志 - 修真世界</title>
</head>

<body>
  <h1>Python</h1>
  <p>一门简介、快速、易用的编程语言。</p>
  <a href="https://www.itcast.cn">传智教育-黑马程序员</a>
</body>
</html>
```

lxml库

仙逆人物志 - 修真世界

Python

一门简介、快速、易用的编程语言。

传智教育-黑马程序员

<https://www.itcast.cn>

- lxml: 是一个高性能的HTML/XML文档的解析库，支持基于XPath语法来解析和获取网页数据。
- 安装: `pip install lxml`

```
from lxml import html

# 加载html文件
with open("resources/仙逆人物志.html", "r", encoding="utf-8") as f:
    html_content = f.read()

# 解析网页内容
doc = html.fromstring(html_content)
th_list = doc.xpath("//table/thead/tr[1]/th/text()")
print(th_list)

td_list = doc.xpath("//table/tbody/tr[1]/td/text()")
print(td_list)
```

Xpath: 是一种用于在HTML/XML文档中导航或定位元素的查询语言，让你能够准确的定位文档中的特定元素、属性或文本。

```
from lxml import html

with open("resources/csplayer.html", "r", encoding="utf-8") as f:
    # 读取html代码
    html_doc = f.read()
    # 解析为文本, 转换为文本对象
    html_txt=html.fromstring(html_doc)

    # 解析表头
    head_list=html_txt.xpath("//table/thead/tr/th/text()") #和目录不同, 这走到哪为定位到哪, 返回的就是哪
    print(head_list)

    # 解析表格第一行数据
    table_first=html_txt.xpath("//table/tbody/tr[1]/td/text()")
    print(table_first)

    # 获取所有行数据
    table_all=html_txt.xpath("//table/tbody/tr") #返回的是所有的tr对象组成的列表
    for tr in table_all: #每一个tr
        tr_list=tr.xpath("./td/text()") #获取tr下的所有td内容
        print(tr_list) string
```

xpath 是定位,与目录不同; tbody/tr 则返回的是 tr 对象组成的列表,而不是 tr 中的对象列表;

1.3.1 xpath 语法

Xpath语法

● Xpath: 一种在HTML/XML文档中导航或定位元素的查询语言, 让你能够准确的定位文档中的特定元素、属性或文本。

表达式	描述	样例
/	从根节点的直接子元素	/html/body/div/h1
//	从任意位置选择节点	//h1
.	当前节点下查找	./a 与 ./a
[n]	选择第n个元素	//p[2]
[last()]	选择最后一个元素	//p[last()]
[@attr]	选择有该属性的元素	//p[@color]
[@attr='value']	选择该属性值等于指定值的元素	//p[@color='red']
*	匹配任何元素节点	//body/div/*
@*	匹配元素的任何属性	//body/div/a/@*
text()	获取文本内容	//div/p/text()

```
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>仙逆人物志 - 修真世界</title>
</head>
<body>
  <div>
    <h1>Python</h1>
    <p>一门简介、快速、易用的编程语言。</p>
    <p>人生苦短, 我用Python。</p>
    <p color="red">AI大模型开发、AI智能应用开发。</p>
    <a href="https://www.itcast.cn">黑马程序员</a>
  </div>
</body>
</html>
```

~~./从前面节点开始,往下的儿子找, //从前面节点开始,往下的任意子孙找,~~

```
a_list = document.xpath("//td/img/@src")
print(a_list)
```

表示获取 img 标签下的属性值;
而 img[@src]表示获取带有 src 属性的 img 对象;

1.3.2 入门完整案例

```
import requests
from lxml import html
# 设置目标url
target_url="https://www.tiobe.com/tiobe-index/"

#发出请求并获得数据
response=requests.get(target_url)
#解析数据为文本
doc=html.fromstring(response.text)

#提取表头
th_list=doc.xpath("/html/body/section/div/article/table[1]/thead/tr/th/text()")
print(th_list)

#提取每行数据
tr_list=doc.xpath("//*[id='top20']/tbody/tr")#获取到所有行(tr对象)
for tr in tr_list:#遍历每个行的tr对象
    td_list=tr.xpath("./td/text()")#获取每行中每个值的列表
    print(td_list)
```

自动查找法:

- 1>浏览器 f12,控制台左上角定位器;
- 2>鼠标直到哪个数据,源代码定位到哪个数据;
- 3>右键取消定位器;
- 4>到源代码位置右键复制 xpath 路径;
- 5>根据单个数据的路径,获取想爬的数据路径;

1.4 csv 文件

CSV

- **CSV: (Comma-Separated Values, 逗号分隔值)**，是一种简单、通用的文本文件格式，用于存储表格数据，可以直接使用Excel打开。

方式一

```
with open("resources/01.csv", "w", encoding="utf-8") as f:
    f.write("姓名,年龄,性别,爱好\n")
    f.write("王林,29,男,Python,Java\n")
    f.write("红蝶,18,女,Java\n")
```

方式二

```
import csv
with open("resources/02.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["姓名", "年龄", "性别", "爱好"])
    writer.writeheader()
    writer.writerow({"姓名": "王林", "年龄": 29, "性别": "男", "爱好": "Python,Java"})
    writer.writerow({"姓名": "红蝶", "年龄": 18, "性别": "女", "爱好": "Java"})
```

```
# csv操作 - 方式一:
# 写
with open("csv_data/01.csv", "w", encoding="utf-8") as f:
    f.write("姓名,年龄,性别,爱好\n") # 写入表头
    f.write("小王,18,男,'football,Java'\n") # 写入数据
    f.write("小李,18,女,Python\n")
    f.write("小张,18,男,C++\n")
    f.write("小王,20,男,Go\n")

# 读
with open("csv_data/01.csv", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

excel用系统编码;文本文件保存为csv文件时选择anti(即按系统编码);格式编码和打开工具的编码相同时才不会出现乱码;

当同一表项有多个值时,用单引号括起来;

```
import csv

# 写csv文件
with open("resources/test.csv", "w", encoding="utf-8", newline="") as f: # newline去除空行
    writer = csv.DictWriter(f, fieldnames=["姓名", "年龄", "性别", "爱好"]) # 传入文件, 表头
    writer.writeheader() # 写表头
    writer.writerow({"姓名": "张三", "年龄": 18, "性别": "男", "爱好": "football, java"}) # 写每行, 以dict格式
    writer.writerow({"姓名": "张三", "年龄": 18, "性别": "男", "爱好": "football"})
    writer.writerow({"姓名": "张三", "年龄": 18, "性别": "男", "爱好": "football"})
    writer.writerow({"姓名": "张三", "年龄": 18, "性别": "男", "爱好": "football"})
    writer.writerow({"姓名": "张三", "年龄": 18, "性别": "男", "爱好": "football"})
    writer.writerow({"姓名": "张三", "年龄": 18, "性别": "男", "爱好": "football"})

# 读csv文件
with open("resources/test.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f) # reader为迭代器
    for row in reader: # 每行为字典
        print(row)
```

2.0 正则表达式

正则表达式

- **正则表达式 (Regular Expression)** 是一种由特定语法规则组成的文本模式，用来描述、匹配字符串中符合特定规则的字符序列。就相当于一种模式匹配工具，允许用户通过简介的语法进行复杂文本的搜索、匹配、提取和替换工作。

`1[3-9]\d{9}`

18809091231是我的手机号，你记住了吗？我的另一个手机号是18800001266，QQ号是1779989922你记住了吗？

```
import re

s = "我的手机号是18809091231你记住了吗？我的另一个手机号是18800001266，QQ号是1779989922你记住了吗？"

result = re.match(r"1[3-9]\d{9}", s) # match - 从字符串的开头开始匹配 (返回Match对象)
print(result.group())

result = re.search(r"1[3-9]\d{9}", s) # search - 从任意位置开始，搜索第一个匹配项 (返回Match对象)
print(result.group())

result = re.findall(r"1[3-9]\d{9}", s) # findall - 从任意位置开始，搜索所有匹配项 (返回列表)
print(result)
```

正则模块:re

r 的含义:指后面的字符串中的“\”为字符串本身,而非转义

1:第一位数字为 1 [3-9]第二位数字在 3-9 之间 \d 表示 0-9 的数字{9}代表/d 出现 9 次;

```
import re

s1="18809090000是我的手机号，你记住了吗？我的另一个手机号是18966534617"
s2="18091377926是我的手机号，你记住了吗？我的另一个手机号是15389491992，我的另一个手机号是134567896532"

#match: 从字符串开头开始匹配第一个匹配项，返回match对象
result=re.match( pattern: r"1[3-9]\d{9}",s1)
#match的四个常用方法:
print(result.group())#18809090000
print(result.span())#(0, 11)
print(result.start())#0
print(result.end())#11

#search: 从字符串任意位置 开始匹配第一个匹配项，返回match对象
result1=re.search( pattern: r"1[3-9]\d{9}",s2)
print(result1.group())#18091377926
print(result1.span())#(0, 11)
print(result1.start())#0
print(result1.end())#11

#findall: 返回所有匹配项，返回列表
result3=re.findall( pattern: r"1[3-9]\d{9}",s2)
print(result3)#[ '18091377926', '15389491992', '13456789653']
```

2.1 正则表达式语法

正则表达式-语法

1[3-9]\d{9}

- 正则表达式的写法多种多样，具体的语法如下：

表达式	描述	表达式	描述
普通字符	字母、数字、汉字及大多数字符，直接匹配自身	*	出现任意次（0次或无数次）
.	匹配任意一个字符（除\n）	+	至少出现1次（1次或无数次）
\d	匹配数字 0-9	?	至多出现一次（0次或1次）
\D	匹配非数字	{m}	出现m次
\w	匹配单词字符，即a-z、A-Z、0-9、_、其他语言字符	{m,}	至少出现m次
\W	匹配非单词字符	{m,n}	出现m到n次
[aeiou]	匹配其中的任何单个字符		或的意思，匹配左右任意一个表达式
[^aeiou]	求反，匹配不在字符列表中的任何单个字符	()	分组，将括号里的多个字符视为一个单元
[0-5]	表示范围	^	匹配字符串开头
		\$	匹配字符串结尾

1[3-9]\d*

1[3-9]\d+

1[3-9]\d?

1[3-9]\d{9}

1[3-9]\d{9,}

1[3-9]\d{9,11}

1(3|5|7|8|9)\d{9}

ab+

(ab)+

^1[3-9]\d{9}\$

`^1[3-9]\d{9}$` 表示必须 1 开头,9 位数字结尾;

```
s2 = "现在的时间是2026-02-06 10:05:25, 今天的天气还可以, 气温是28度 "  
print(re.findall( pattern: r"\d{4}-\d{2}-\d{2}", s2))  
print(re.findall( pattern: r"(\d{4})-(\d{2})-(\d{2})", s2))
```

```
['2026-02-06']  
[('2026', '02', '06')]
```

分组操作:整体符合规则,但返回其中每个括号的个体;